Programming Fundamentals LAB – Spring 2026

Lab # 5

(BS-IT/AI-F25 Morning & Afternoon)

Submission Deadline: ***Saturday, March 7, 2026. (till 12:00 PM)***

Instructions:

- You must complete all tasks individually. Absolutely NO collaboration is allowed. Any traces of plagiarism/cheating or usage of AI tools will result in an **"F"** grade in this course and lab.
- **Indent** your code properly.
- Use meaningful variable and function names. Use the **camelCase** notation.
- Use meaningful prompt lines/labels for all input/output.
- Absolutely No tolerance policy once time is up.
- Late submissions will NOT be accepted, whatever your excuse may be.

Important Note:

We need the files of tasks you have done in a folder that you will submit in zip as ***Bitf25-A001 or Baif25-M001***

Task # 1

Write a C++ program that:

1. Asks the user to enter a **positive number**.
2. Repeatedly **halve the number** (divide by 2) if it is **even**, or **subtract 1** if it is **odd**.
3. Print each number in the process.
4. Stop when the number becomes **0**.
5. Count and display the **total steps** taken.

Use **do-while loop** and **if-else** only..

Sample Input: 10
Sample Output:
Sequence: 10 5 4 2 1 0
Total steps: 5

Task # 2

Write a C++ program for a **Number Guessing Game**.
Your program should:

1. Randomly select a number between **1 and 50**.
2. Ask the user to guess the number.
3. After each guess, give feedback:
    - If the guess is too high, print: "Too high! Try again."
    - If the guess is too low, print: "Too low! Try again."
4. Keep asking until the user guesses the correct number.
5. When the user guesses correctly, print:
   "Congratulations! You guessed the number in X tries."
   where X is the number of attempts.
6. Use only **while or do-while loops** and **if-else statements**.
7. Do **not** use arrays or functions.

Sample :

Guess the number (1-50): 30
Too low! Try again.

Guess the number (1-50): 45
Too high! Try again.

Guess the number (1-50): 38
Congratulations! You guessed the number in 3 tries.

Task # 3

Write a C++ program that:

1. Ask the user to enter **total available tickets** in the cinema.
2. Then repeatedly ask the user **how many tickets they want to book**.
3. For each request:
    - If the requested number is **less than or equal to available tickets**, approve the booking and reduce the available tickets.
    - If the requested number is **more than available tickets**, print "Not enough tickets! Try again."
4. Continue this process using a **do-while loop** until all tickets are sold out (available tickets = 0).
5. Display how many bookings were successfully made and how many tickets were left (should be 0 at the end).

Use only **do-while loop** and **if-else**.

Sample Input: Enter total tickets: 10

Sample Output:
Enter number of tickets to book: 3
Booking successful! Tickets left: 7

Enter number of tickets to book: 5
Booking successful! Tickets left: 2

Enter number of tickets to book: 4
Not enough tickets! Try again.

Enter number of tickets to book: 2
Booking successful! Tickets left: 0

All tickets sold out!
Total successful bookings: 3

Task # 4

Write a C++ program that:

- Takes an integer n from the user.
- Prints the first n Fibonacci numbers.
- Use only **for loop**.

Sample Input:

Enter number of terms: 5

Sample Output: 0 1 1 2 3

Task # 5

A student is participating in a **30-day Reading Challenge**.

On the first day, the student reads **1 page**.
On the second day, the student reads **1/2 page**.
On the third day, the student reads **1/3 page**, and so on.

This pattern continues until the **nth day**, where the student reads **1/n pages**.

Write a C++ program that:

- Takes an integer n (number of days) from the user.
- Calculates the total number of pages read using the series:

$$1 + 1/2 + 1/3 + 1/4 + \ldots + 1/n$$

- Display the final total (use floating-point variables).
- Use a **for loop**

Sample Input: Enter number of days: 4

Sample Output: Total pages read: 2.08333

Task # 6

A student is eating snacks in a **fun challenge**:

- On the **1st snack**, the student eats **1 sweet**
- On the **2nd snack**, the student eats **2 salty chips** but counts it as negative
- On the **3rd snack**, eats **3 sweets** (+3)
- On the **4th snack**, eats **4 salty chips** (-4)

This continues up to **n snacks** in total.

Write a C++ program that:

1. Takes an integer n (total number of snacks) from the user.
2. Calculates the **net score** using the alternate series:

$$1 - 2 + 3 - 4 + 5 - 6 \ldots N$$

Sample Input: Enter number of snacks: 5

Sample Output: Net score: 3

Task # 7

Create a program that helps a user **track monthly savings**.
The program should repeatedly display this menu:
1. Calculate Savings for the Month
2. Exit

- If the user selects **1**:
  1. Ask for **monthly income**.
  2. Ask for **monthly expense per week**.
  3. Use a **for loop** to calculate **total expenses for 4 weeks**.
  4. Subtract total expenses from income and display **remaining savings**.
- Use a **do-while loop** to repeat the menu until the user selects **2 (Exit)**.

Sample Input:
Enter choice: 1
Enter monthly income: 4000
Enter weekly expense: 800

Sample Output:
Total expenses for 4 weeks: 3200
Remaining savings: 800

Task # 8

A game has **N players**. Each player scores some points.

Write a program that:

1. Takes **N scores** from the user (one by one).
2. Finds the **highest score** and the **second highest score**.
3. Print both scores at the end.

Use only **loops and if-else**. No arrays or functions required.

Sample Input:

Enter number of players: 5
Enter score for player 1: 10
Enter score for player 2: 25
Enter score for player 3: 15
Enter score for player 4: 20
Enter score for player 5: 5

Sample Output :

Highest score: 25
Second highest score: 20

Task # 9

Write a program that:

1. Takes a positive integer N from the user.
2. Prints a **pyramid of stars** using a **for loop**:

Sample Input 1: 4

Sample Output:

```
   *
  **
 ***
****
```

Task # 10

You are climbing a staircase. It takes **n steps** to reach the top.
Each time you can either:

- climb **1 step**, or
- climb **2 steps**

Write a C++ program that:

1. Takes an integer n as input from the user.
2. Calculates the total number of **distinct ways** to reach the top.
3. Displays the total number of possible ways.

NOTE: Use only:

- loops (while or for)
- variables
- if-else

Sample Input : Enter number of steps: 2
Sample Output : Total distinct ways:2

Sample Input: Enter number of steps: 4

Sample Output 2: Total distinct ways: 5.

Task # 11

Write a program that:

1. Takes a positive integer N from the user.
2. Prints all numbers between **1 and N** that are **prime**.
3. Use **loops and if-else** only.

Sample Input: Enter a number: 10
Sample Output: Prime numbers between 1 and 10: 2 3 5 7

Task # 12

A high-security digital vault uses a 4-digit PIN (d_1 d_2 d_3 d_4) that must follow a specific "Zig-Zag" mathematical pattern to be valid. Your task is to write a program that takes a 4-digit integer as input and determines if it is a Valid Zig-Zag PIN.

The Rules of the Lock:

1. Zig-Zag Pattern:

- The second digit must be greater than the first ($d_1 < d_2$).
- The third digit must be smaller than the second ($d_2 > d_3$).
- The fourth digit must be greater than the third ($d_3 < d_4$).

2. No Zeroes:

- To be a valid security code, the number cannot contain the digit 0.

Your Program Must:

- Ask the user to enter a 4-digit integer.
- Extract the individual digits using arithmetic operators (/ and %).
- Check if the Zig-Zag conditions are met.
- Print "VALID PIN" or "INVALID PIN" with a reason for the failure.

Sample Input: Enter 4-digit PIN:1426
Sample Output:
Success: 1426 follows the Zig-Zag pattern.
RESULT: VALID PIN